



Cyberster Internship Program

Internship Report

WEEK 3: SURICATA IDS INTEGRATION WITH WAZUH

Track: Blue Team

Name: Haris Zamir

Roll Number: CSI-B1-487

Instructor: Abdullah Zia

Overview

Week 3 concentrated on moving from simple host-based monitoring to Network Security Monitoring (NSM) and automated threat response. The main goal was to set up Suricata as an independent Network Intrusion Detection System (NIDS) on the Kali sensor, connect its generated telemetry to the Wazuh SIEM, and configure an Active Response mechanism capable of automatically “shunning” or blocking attackers in real time using signature-based detections.

Day 15-16: Suricata Installation and Configuration

Objective: To deploy Suricata and tune its packet capture engine for the virtualized lab environment.

I installed the Suricata package and completed the initial setup inside the `suricata.yaml` configuration file. One important task was determining the appropriate network interface (`eth0` on the Host-Only adapter) so that Suricata could properly monitor the traffic flowing between the Attacker machine (Kali) and the Target system (Ubuntu).

In addition, I configured the `AF_PACKET` capture method, enabling Suricata to interact directly with the Linux kernel’s packet ring buffer for efficient, high-speed packet inspection while minimizing packet loss during heavy traffic simulations.

The interface variable was manually assigned to the corresponding physical network adapter within the YAML configuration file.

```
# Linux high speed capture support
af-packet:
  - interface: eth1
    # Number of receive threads. "auto" uses the number of cores
    #threads: auto
    # Default clusterid. AF_PACKET will load balance packets based on flow.
    cluster-id: 99
```

The service is operating in IDS mode through systemctl, and the engine successfully initialized and loaded the signature database.

```
--$ sudo systemctl status suricata
suricata.service - Suricata IDS/IDP daemon
Loaded: loaded (/usr/lib/systemd/system/suricata.service; disabled; preset: disabled)
Active: active (running) since Thu 2026-03-26 12:37:52 PKT; 37s ago
Invocation: 1a161dde20464d6187890ddb8f81876de
Docs: man:suricata(8)
      man:suricatasc(8)
      https://suricata.io/documentation/
Main PID: 21902 (Suricata-Main)
Tasks: 8 (limit: 4500)
Memory: 36.9M (peak: 37.2M)
CPU: 431ms
CGroup: /system.slice/suricata.service
        └─21902 /usr/bin/suricata -D --af-packet -c /etc/suricata/suricata.yaml --pidfile /var/run/suricata/suricata.pid

Mar 26 12:37:51 kali systemd[1]: Starting suricata.service - Suricata IDS/IDP daemon...
Mar 26 12:37:52 kali suricata[21901]: i: suricata: This is Suricata version 8.0.4 RELEASE running in SYSTEM mode
Mar 26 12:37:52 kali systemd[1]: Started suricata.service - Suricata IDS/IDP daemon.
```

Day 17: Suricata Rule Writing & Signature Engineering

The objective of Day 17 was to move beyond relying on default community rulesets and begin performing Signature Engineering. This process involved studying specific attack scenarios, recognizing unique network behavior patterns, and creating accurate detection logic for identifying Nmap reconnaissance activity, HTTP exploitation attempts, and high-volume DoS attacks.

I created a custom ruleset within `/etc/suricata/rules/local.rules`. To prevent conflicts with the existing Emerging Threats community rules, I used unique Signature IDs (SIDs) within the 9,000,000 range.

Rule 1: Nmap NULL Scan Detection

- Scenario: Detects stealth reconnaissance techniques where an attacker sends TCP packets without any flags enabled in an attempt to evade basic stateful firewalls.
- Logic: I applied the flags:0; and ack:0; conditions to specifically identify packets containing an empty TCP flag header, which is characteristic of a NULL scan.

Rule 2: HTTP Directory Traversal Attempt

- Scenario: Detects attempts to gain unauthorized access to sensitive system files such as `/etc/passwd` by exploiting directory traversal vulnerabilities.
- Logic: I used the `http.uri` and `content:"../"` keywords to examine the URI buffer for repeated traversal patterns.

Rule 3: ICMP (Ping) Flood Detection

- Scenario: Identifies possible Denial of Service (DoS) attacks where a target system is flooded with rapid ICMP echo requests.
- Logic: Since a single ICMP packet is considered legitimate traffic, I implemented the threshold keyword so the rule would only trigger when 20 packets from the same source are observed within a 10-second interval (type both, count 20, seconds 10).

This screenshot shows the full syntax of the three custom rules, including the rule headers (alert tcp/http/icmp), traffic direction operators (->), and the options section containing fields such as msg and sid.

```
GNU nano 8.6 /etc/suricata/rules/local.rules *
# Rule 1: Detects an Nmap NULL scan by looking for TCP packets with no flags set.
# This is a classic scanning technique used to bypass simple firewalls.
alert tcp any any -> $HOME_NET any (msg:"CUSTOM Nmap NULL Scan Detected"; flags:0; ack:0; sid:9000001; rev:1;)

# Rule 2: Detects directory traversal attempts in HTTP URIs.
# Specifically looks for the "../" string which indicates an attempt to access sensitive system files.
alert http any any -> $HOME_NET any (msg:"CUSTOM HTTP Directory Traversal Attempt"; http.uri; content:"../"; sid:9000002; rev:1;)

# Rule 3: Detects ICMP (Ping) Flood attacks.
# Uses a threshold to trigger only if more than 20 pings arrive from a single source within 10 seconds.
alert icmp any any -> $HOME_NET any (msg:"CUSTOM ICMP Flood Detected"; itype:8; threshold: type both, track by_src, count 20, seconds 10; sid:9000003; rev:1;)
```

This screenshot shows the live output from /var/log/suricata/fast.log. Each log entry verifies that the custom rules were triggered successfully, displaying details such as the custom alert message (for example, "CUSTOM Nmap NULL Scan Detected"), the event timestamp, and the source IP address (Kali attacker machine) along with the destination IP address (Ubuntu Manager target system).

```
$ sudo tail -n 10 /var/log/suricata/fast.log
03/26/2026-12:46:55.166414 [**] [1:2022973:1] ET INFO Possible Kali Linux hostname in DHCP Request Packet [**] [Classification: Potential Corporate Privacy Violation] [Priority: 1] {UDP} 192.168.19.129:68 -> 192.168.19.254:67
03/26/2026-12:53:43.010814 [**] [1:2100498:7] GPL ATTACK_RESPONSE id check returned root [**] [Classification: Potentially Bad Traffic] [Priority: 2] {TCP} 217.160.0.187:80 -> 192.168.19.129:54596
03/26/2026-12:55:08.432701 [**] [1:2100498:7] GPL ATTACK_RESPONSE id check returned root [**] [Classification: Potentially Bad Traffic] [Priority: 2] {TCP} 217.160.0.187:80 -> 192.168.19.129:40858
```

Days 18 & 19: SIEM Integration & Unified Telemetry

Objective: To centralize network security alerts within the Wazuh Manager for improved cross-layer event correlation.

- **Log Ingestion:** I configured the Wazuh Agent on the Kali system to function as a log forwarder. By directing the ossec.conf collector to Suricata's eve.json file in EVE JSON format, I ensured that detailed metadata such as flow IDs, GeoIP information, and payload excerpts were forwarded to the Wazuh Manager.
- **Decoder Analysis:** I confirmed that the Wazuh Manager's built-in decoders were successfully interpreting the Suricata JSON structure, enabling the SIEM platform to properly classify the events as "Network IDS" alerts.

This screenshot displays the XML <localfile> configuration, where the json log format is specified to retain Suricata's detailed metadata during log ingestion.

```
#custom suricata
<localfile>
  <log_format>json</log_format>
  <location>/var/log/suricata/eve.json</location>
</localfile>
```

This screenshot displays the Wazuh WUI Dashboard, confirming the successful integration and communication “handshake” between the Network Intrusion Detection System (NIDS) and the SIEM platform.

Mar 26, 2026 @ 13:20:27.883 001 Ubuntu-Manager

Table JSON Rule

```
{
  "agent": {
    "ip": "192.168.19.129",
    "name": "Ubuntu-Manager",
    "id": "001"
  },
  "manager": {
    "name": "ubuntu-manager"
  },
  "data": {
    "app_proto": "tls",
    "ip_v": "4",
    "in_iface": "eth1",
    "src_ip": "192.168.19.130",
    "src_port": "443",
    "event_type": "alert",
    "alert": {
      "severity": "3",
      "signature_id": "2210016",
      "rev": "2",
      "gid": "1",
      "signature": "SURICATA STREAM CLOSEWAIT FIN out of window",
      "action": "allowed",
      "category": "Generic Protocol Command Decode"
    },
    "flow_id": "598254169790031.000000",
    "dest_ip": "192.168.19.1",
    "proto": "TCP",
    "dest_port": "62054",
    "pkt_src": "wire/pcap",
    "flow": {
      "src_ip": "192.168.19.1",
      "src_port": "62054",
      "pkts_to_server": "8",
      "dest_ip": "192.168.19.130",
      "start": "2026-03-26T13:20:26.401435+0500",
      "bytes_to_client": "1775",
      "bytes_to_server": "2263",
      "pkts_to_client": "5",
      "dest_port": "443"
    },
    "timestamp": "2026-03-26T13:20:26.453693+0500",
    "direction": "to_client"
  },
  "rule": {
    "firedtimes": 88,
    "mail": false,
    "level": 3,
    "description": "Suricata: Alert - SURICATA STREAM CLOSEWAIT FIN out of window",
    "groups": [
```

Days 20 & 21: Active Response & Automated Remediation

Objective: To complete the “Detection-to-Response” cycle by automatically blocking malicious IP addresses.

I configured an Active Response policy so that when Suricata detects malicious activity, such as an Nmap scan, it generates an alert that is processed by the Wazuh Manager. The Manager then executes the firewall-drop script, which dynamically adds a DROP rule into the Linux kernel firewall to block the offending source IP in real time.

During implementation, I encountered an issue where the script failed with the error: “Cannot read ‘srcip’ from data.” This was resolved by modifying the `<expect>` tag in the manager configuration, ensuring that the script correctly extracted the attacker’s IP address from the Suricata alert payload.

The final, debugged `ossec.conf` showing the level-based trigger logic.

```
command>
  <name>firewall-drop</name>
  <executable>firewall-drop</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>

<!--

<active-response>
  active-response options here
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>100002</rules_id> <timeout>300</timeout>

</active-response>
-->
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_group>suricata</rules_group>
  <timeout>600</timeout>
</active-response>
```

Shows the Kali terminal where the **ping** has timed out. This is the physical proof that the automated block is preventing the attacker from communicating with the target.

```
└─$ ping 192.168.19.130
PING 192.168.19.130 (192.168.19.130) 56(84) bytes of data.
```

Displays the output of **sudo iptables -L -n**. This shows the specific **DROP** entry for **192.168.19.129** in the **FORWARD** chain, confirming the firewall has "shunned" the attacker.

```
DROP      0    --  192.168.19.129    0.0.0.0/0
DROP      0    --  192.168.19.129    0.0.0.0/0
```

Summary

This report provides a detailed technical audit of the NIDS and Active Response integration completed during Week 3. Each stage of the deployment process, beginning with the installation and configuration of Suricata and ending with the final iptables enforcement actions, has been thoroughly documented using technical evidence and log analysis.

The screenshots included in this document confirm that the complete Detection-to-Response workflow is functioning correctly and that the Wazuh Manager is successfully correlating external network telemetry from the NIDS environment. By resolving issues related to JSON field mapping and firewall chain persistence, the report demonstrates that the environment has achieved a stable and properly secured baseline.

All configuration samples, technical proofs, and troubleshooting logs have been carefully cross-verified to maintain the accuracy and integrity of the presented data. This documentation serves as a complete record of the automated mitigation mechanisms implemented during the project, as well as the successful restoration of the environment to an "all clean" operational state.

